

VCC Mobile API — Register & My Seva

Hare Krishna Movement Visakhapatnam — the 2 core calls for the volunteer mobile app and
harekrishnavizag.org

Public · No login required

Overview 1. Register a volunteer 2. My Seva lookup

Notes & related calls

Overview

These are the **two core calls** every volunteer interaction depends on. Both are **public** (no login, no token) and both are CORS-enabled for `https://harekrishnavizag.org` and `https://www.harekrishnavizag.org`.

#	Purpose	Endpoint
1	Register — a volunteer signs up for an event (mobile app or the website)	POST <code>/api/events/public/:eventId/register</code>
2	My Seva — a volunteer views their profile and assigned seva by phone	GET <code>/api/volunteers/by-phone/:phone</code>

The phone number is the volunteer's identity. A volunteer who registers from the mobile app is the **same person** in the admin panel as one who registered on the website — the phone number matches them up.

Base URL

Environment	Base URL
Production	<code>https://vcc-client.vercel.app</code>
Local dev	<code>http://localhost:3000</code>

Call 1 — Register a volunteer for an event

POST /api/events/public/:eventId/register

`eventId` is the short uppercase event code (e.g. `SKJ26V`), shown on the events list. The call does three things automatically:

1. Looks up the volunteer **by phone**. If they've volunteered before, their existing profile is reused.
2. If they're new, **creates their profile**.
3. Creates the **registration record** linking them to the event.

Request body (JSON)

```
{
  "name": "Rama Das",
  "phone": "9876543210",
  "age": 28,
  "gender": "male",
  "locality": "Dwaraka Nagar",
  "occupation": "Software Engineer",
  "occupationType": "working",
  "company": "ACME Ltd",
  "skills": ["it", "photography"],
  "photoKey": "volunteers/1725182601234-ab12cd.jpg",
  "serviceAvailability": [{ "date": "2026-09-04", "timeSlot": "Morning 8-11 AM" }],
  "customAnswers": [{ "fieldId": "d4e5f6", "value": "Parking" }],
  "notes": ""
}
```

Required fields: `name` , `phone` — everything else is optional.

Field rules

Field	Rules
<code>phone</code>	Any Indian number. Non-digits are stripped and only the last 10 digits are kept. Invalid → <code>400 "Phone number must be exactly 10 digits"</code> .
<code>gender</code>	<code>male</code> <code>female</code> <code>other</code>
<code>occupationType</code>	<code>student</code> <code>working</code> (use <code>institution</code> for students, <code>company</code> for working)
<code>skills</code>	<code>medical</code> , <code>photography</code> , <code>videography</code> , <code>driving</code> , <code>electrical</code> , <code>sound</code> , <code>it</code> , <code>graphic_design</code> , <code>cooking</code> , <code>crowd_management</code> , <code>other</code>

photoKey	Returned by <code>POST /api/upload/photo</code> (upload first, then pass the key). Required only when the event has <code>photoRequired: true</code> .
serviceAvailability	Array of <code>{ date, timeSlot }</code> matching the event's slots (from <code>GET /api/events/public/:eventId/time-slots</code>).
customAnswers	Array of <code>{ fieldId, value }</code> (an object map like <code>{ "d4e5f6": "Parking" }</code> also works). Required/important custom fields are enforced server-side. Checkbox and <code>devotee_select</code> accept arrays of values.

curl (Windows cmd)

```
curl -X POST https://vcc-client.vercel.app/api/events/public/SKJ26V/register ^
-H "Content-Type: application/json" ^
-d '{"name": "Rama Das", "phone": "9876543210"}'
```

Tip: put the JSON in a file (`body.json`) and use `-d "@body.json"` to avoid quote-escaping pain on Windows.

Response (201 — created)

```
{
  "message": "Registration successful",
  "registration": {
    "_id": "6a796695123edd9fb32a061b",
    "eventId": "6a7717364ae497f56781ce9b",
    "volunteerId": "6a796695123edd9fb32a061a",
    "status": "registered",
    "serviceAvailability": [],
    "customAnswers": []
  },
  "volunteer": {
    "_id": "6a796695123edd9fb32a061a",
    "name": "Rama Das",
    "phone": "9876543210",
    "photoKey": "volunteers/1725182601234-ab12cd.jpg"
  }
}
```

Possible errors

Status	Message	App should show
--------	---------	-----------------

400	Name is required	Fix the name field
400	Phone number must be exactly 10 digits	Fix phone input
400	Registration is not open for this event	"Registrations closed for this event"
400	Registration deadline has passed	"Registration is over"
400	"<label>" is required	Fill in the highlighted custom field
400	"<label>" must be a number / has an invalid option / must be a valid date	Fix the highlighted field
404	Event not found	Wrong event code / event removed
409	Already registered for this event	"You're already registered" (safe to show as a success state)

Alternative: `POST /api/registrations` does the same thing but takes the Mongo `_id` as `eventId` in the body. Prefer the `... /register` URL form — it uses the stable, human-readable event code.

Call 2 — My Seva (profile + assignments by phone)

GET `/api/volunteers/by-phone/:phone`

Given the volunteer's registered phone number, returns their profile plus **all their registrations** with event details, assigned service, and the coordinator to contact. Use it for the "My Seva" screen and for the pre-registration check ("does this phone already exist?").

curl

```
curl https://vcc-client.vercel.app/api/volunteers/by-phone/9876543210
```

Response (200)

```
{
  "volunteer": {
    "_id": "6a796695123edd9fb32a061a",
    "name": "Rama Das",
    "phone": "9876543210",
```

```

    "photoKey": "volunteers/1725182601234-ab12cd.jpg"
  },
  "registrations": [
    {
      "_id": "6a796695123edd9fb32a061b",
      "status": "assigned",
      "eventId": {
        "_id": "6a7717364ae497f56781ce9b",
        "name": "Sri Krishna Janmashtami 2026",
        "eventId": "SKJ26V",
        "status": "registration_open",
        "venue": "Gadiraju convention centre",
        "eventStart": "2026-09-04T04:16:00.000Z",
        "eventEnd": "2026-09-08T04:16:00.000Z"
      },
      "serviceId": {
        "_id": "...",
        "name": "Parking",
        "description": "...",
        "coordinatorId": { "_id": "...", "name": "chaitanya", "phone": "..." }
      }
    }
  ]
}

```

How to use it in the app

- **status** is the volunteer's progress for that event: **registered** → **assigned** → **attended** (also **no_show** / **cancelled**).
- When the admin assigns work, **serviceId** becomes populated (it's **null** /absent until then). Show the service name to the volunteer.
- **serviceId.coordinatorId** is the person to contact for that seva (tap-to-call / WhatsApp).
- **photoKey** is the volunteer's photo, if uploaded. Display it with `GET /api/upload/photo?key=<photoKey>` (302 redirect to the image).

Possible errors

Status	Message	App should show
404	Volunteer not found	"No account found with this number" + offer to register

Same data, shareable link: the `/my-seva/<phone>` web page uses the identical endpoint, so a volunteer can also view their seva in a browser.

Notes & related calls

Statuses are read-only for volunteers

Volunteers can never change their own status. The mobile app only **reads** this data — all status changes (assigning a service, marking attendance, canceling) happen in the admin panel.

Related calls (same public API family)

Endpoint	Purpose
GET /api/events/public	List events a volunteer can register for
GET /api/events/public/:eventId	Event details + custom fields + slots for the registration form
GET /api/events/public/:eventId/time-slots	The availability slots to send back as <code>serviceAvailability</code>
POST /api/upload/photo	Upload the volunteer photo → returns <code>photoKey</code>
GET /api/events/public/:eventId/check-in	Venue attendance lookup (see CHECK-IN-API)
POST /api/events/public/:eventId/check-in	Venue attendance confirm (see CHECK-IN-API)